UNIVERSIDAD TECMILENIO

Sistemas y Tecnologías de Administración del conocimiento

Jesús Clemente Chio Rodríguez

Project Progress

al07004708: Sebastián Soria González
al07026440: Gustavo Andres Ontiveros Ipiña
al03095697: Roberto Abisai Hernández De La Rosa

Table of Contents

Project Progress Documentation 2

1. Description of the fictitious company 3

Main activities 3

Documents and knowledge resources it manages 3

Why a knowledge management system is needed 3

Areas that generate, use or update information 4

2. Knowledge life cycle map 5

Stages explained with one case 5

Tacit and explicit knowledge 5

3. Ontology and conceptual graph of organizational knowledge 7

How the ontology covers the required elements 7

Relations between concepts 8

Examples of instances 8

4. How the prototype works 10

Repository structure 10

Data flow 10

Modules 11

How the search is computed 11

How the classification works 11

5. Evidence of prototype development 13

5.1 Dashboard and input documents 13

5.2 Classification with the repository script 14

5.3 Classification tests with new documents 15

5.4 Semantic search examples 17

5.5 Knowledge base browsing 21

5.6 Classifier evaluation with the repository scripts 22

6. Evidence of practical exercises (Topics 1 to 6) 24

Topic 1 – Knowledge audit 25

Topic 2 – Text preprocessing and TF-IDF 25

Topic 3 – Knowledge representation 26

Topic 4 – Information retrieval 26

Topic 5 – Classification tests 26

Topic 6 – Similarity and clustering 27

7. Repository and reproduction 28

8. Conclusion 29

References 29

Project Progress Documentation

This document presents the progress of the knowledge management project developed for NexaTech Solutions, a fictitious technology company. It describes the company, maps the knowledge life cycle, presents an ontology and conceptual graph of the organizational knowledge, explains how the prototype works, and shows evidence of its execution: screenshots, classification results, semantic search examples and the practical exercises of Topics 1 to 6 that contributed to the project.

The prototype is a Python application that stores technical documents, classifies them automatically and retrieves them with natural-language queries. Source code, dataset, test documents and results are in the repository: https://github.com/Andres145876/NexaTech-Knowledge-Management.

The project at a glance

102	4	92.3%	8 / 8	6
documents in the committed dataset	knowledge categories	hold-out accuracy of the classifier (85.1% in 5-fold cross-validation)	new test documents classified correctly	semantic search examples

All screenshots and figures in this version were produced with the dataset committed to the repository (`DATA/documents.csv`, 102 documents), so the application and the scripts show the same numbers.

1. Description of the fictitious company

NexaTech Solutions is a fictitious mid-sized Latin American company with about 400 employees. It builds custom software and sells managed IT services, which include cloud hosting, connectivity, information security and 24/7 technical support. Its clients are banks, retailers and public agencies.

Main activities

- **Technical support:** agents solve client incidents and accumulate fixes, workarounds and knowledge of client environments.
- **Software development:** engineers produce architectural decisions, code, technical designs and development guidelines.
- **Infrastructure operations:** network and cloud engineers monitor services, plan capacity and handle outages.
- **Cybersecurity:** analysts define security policies, control access to accounts and respond to security incidents.
- **Management:** leaders use reports, contracts and client feedback to set priorities and investments.

Documents and knowledge resources it manages

The daily work of the company produces policies, procedures, technical guidelines, configuration standards, capacity plans, runbooks, support guides and incident reports. In the prototype these documents are organized in four knowledge categories: Cybersecurity, Infrastructure, Software Development and Technical Support.

Why a knowledge management system is needed

Knowledge is spread across tickets, repositories and people's experience, so employees lose time looking for information and key know-how depends on a few specialists. A knowledge management system centralizes the documents, classifies them by category and lets employees find them with natural-language queries, which shortens response times and reduces the risk of losing knowledge.

Areas that generate, use or update information

Area	Generates	Uses	Updates
Technical Support	Ticket notes, fixes, workarounds, account-support guides	Runbooks, policies, release notes	Knowledge base articles that failed in practice
Software Development	Code, designs, development guidelines	Escalated bugs, logs, client environment data	API documentation, guidelines, release notes
Infrastructure	Network policies, capacity plans, runbooks, incident reports	Release notes, monitoring alerts	Runbooks, configuration standards, maintenance schedules
Cybersecurity	Security policies (e.g., multi-factor authentication), response procedures	Incident reports, access logs	Policies and response procedures after incidents
Management	Priorities, SLA targets, staffing decisions	Dashboards, trend reports, contracts	Priorities and client commitments

2. Knowledge life cycle map

Knowledge at NexaTech moves through six stages: **creation and capture**, **storage and documentation**, **organization and classification**, **sharing and transfer**, **application**, and **update and continuous improvement**. These stages expand the capture, sharing and application cycle described by Dalkir (2017). Figure 1 shows, for each stage, the tacit and explicit knowledge involved, the SECI conversion (Nonaka & Takeuchi, 1995), the component of the prototype that supports it and the data in the system.

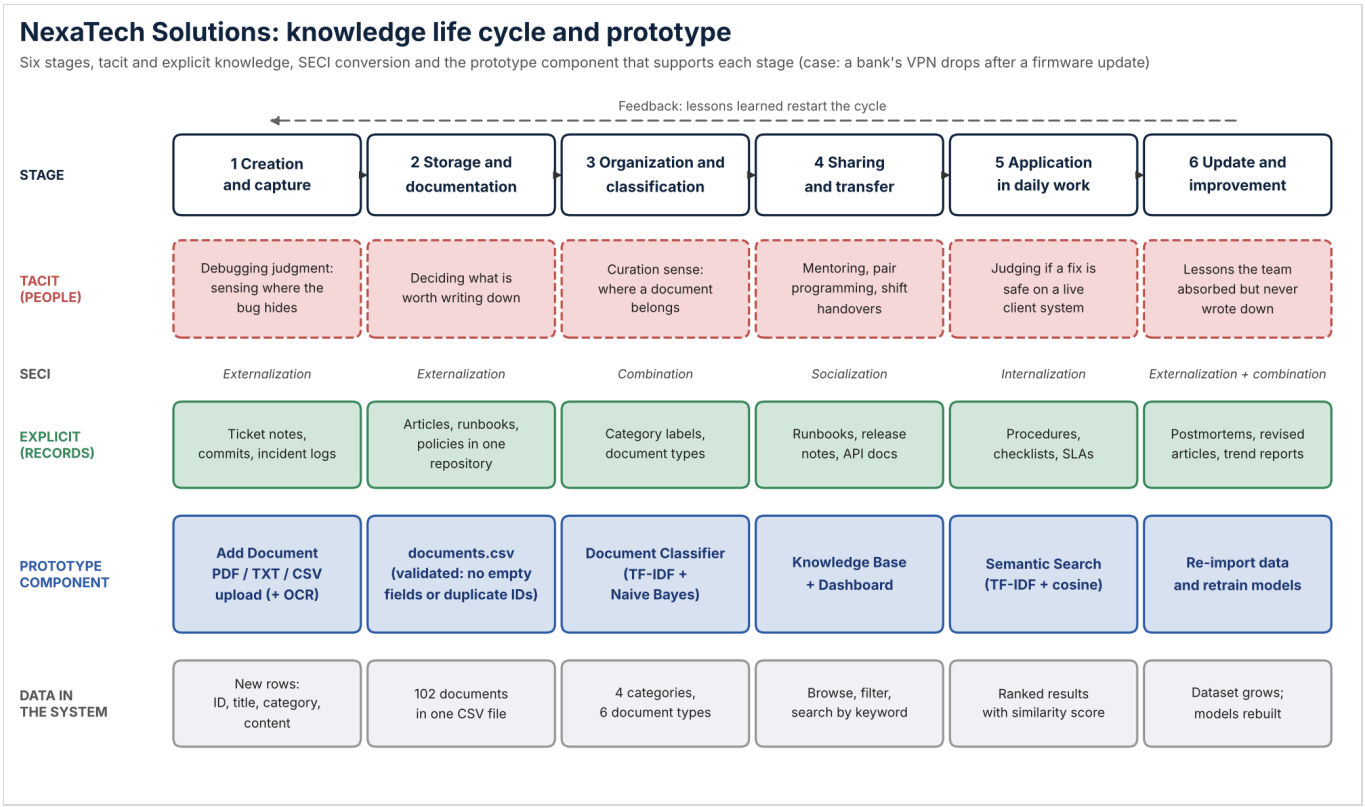


Figure 1. Knowledge life cycle at NexaTech Solutions and its relation to the prototype. Source: own elaboration (DIAGRAMS/lifecycle_map.py).

Stages explained with one case

A bank's VPN connection drops after every firmware update. The case follows the cycle as shown below.

Stage	What happens (VPN case)	Prototype support
1. Creation and capture	A support agent notices the pattern. With an operations engineer she finds that the update resets a tunnel setting and records the cause and workaround in the ticket.	Add Document receives the new document (manual entry, or PDF, TXT or CSV file; OCR for scanned PDFs).
2. Storage and documentation	The workaround is written as an article with a title and content and saved in the central repository.	Saved in `documents.csv`; the app validates required fields and rejects duplicated IDs or contents.
3. Organization and classification	The article is assigned to a knowledge category and tagged by product, client sector and error type.	The Document Classifier (TF-IDF + Naive Bayes) suggests the category (Technical Support); the author confirms it.
4. Sharing and transfer	The article is published; a senior agent mentors a junior one and development ships the fix in the next release.	Knowledge Base and Dashboard make the documents visible and filterable.
5. Application	Two weeks later another client reports the same symptom; an agent finds the article and restores the service in minutes.	Semantic Search ranks the documents by similarity: the query “the VPN connection drops after the firmware update” returns *VPN Connectivity Troubleshooting* first (Figure 18).
6. Update and continuous improvement	The postmortem leads to an automated configuration check, and the article is corrected.	New or corrected documents are imported and the search index and classifier are rebuilt automatically.

Tacit and explicit knowledge

Tacit knowledge lives in people (experience, skill and judgment), while explicit knowledge is written in documents, repositories and databases (Polanyi, 1966). The organization learns as it converts one into the other: socialization (tacit to tacit), externalization (tacit to explicit), combination (explicit to explicit) and internalization (explicit to tacit). The prototype works on the explicit side: it stores documents and makes them easy to classify and retrieve, so that other employees can internalize them.

3. Ontology and conceptual graph of organizational knowledge

The ontology describes the main concepts of the domain and how they relate. It has four layers: the organization and its areas; the knowledge categories that each area generates; the technical documents, which have a document type (policy, procedure, guideline, plan or runbook, support guide, incident report) and belong to one category; and the models and users that classify and retrieve them (classifier, search engine, employee and query). Dashed boxes are real documents from the dataset, shown as examples of instances; the green arrow between DOC007 and DOC003 shows the relation between documents (cosine similarity). The graph is Figure 2, on the landscape page that follows the tables.

How the ontology covers the required elements

Element	Representation in the NexaTech ontology
Main concepts of the domain	Organization (NexaTech Solutions), areas (Technical Support, Software Development, Infrastructure, Cybersecurity), knowledge categories, employee, natural-language query, classifier and search engine.
Document types	Policy, procedure, guideline, plan / runbook, support guide and incident report, all subtypes of *Technical document* (attributes: document_id, title, category, content).
Relationships between concepts	*has area*, *generates / owns*, *asks*, *is processed by*, *subtype of*.
Relationships between documents and concepts	*contains* (category → document), *has type* (document → type), *is similar to* (document ↔ document), *ranks / retrieves* (search engine → document), *reads / assigns category* (classifier → document / category). The Topic 3 exercise (section 6) populates these relations with all 102 documents.

Relations between concepts

Subject	Relation	Object
NexaTech Solutions	has area	Technical Support, Software Development, Infrastructure, Cybersecurity
Area	generates and owns	Knowledge category with its documents
Knowledge category	contains	Technical document
Technical document	has type	Document type: policy, procedure, guideline, plan / runbook, support guide, incident report
Document type	is a subtype of	Technical document (attributes: document_id, title, category, content)
Technical document	is similar to	Other technical documents (cosine similarity of TF-IDF vectors)
Employee	asks	Natural-language query
Natural-language query	is processed by	Search engine (TF-IDF + cosine similarity), which ranks and retrieves technical documents
Classifier (TF-IDF + Naive Bayes)	reads the text of / assigns	A technical document / its knowledge category

Examples of instances

Document	Title	Type	Category
DOC059	Multi-Factor Authentication Policy	Policy	Cybersecurity
DOC003	Network Monitoring Policy	Policy	Infrastructure
DOC007	Network Capacity Planning	Plan / runbook	Infrastructure
DOC099	Backend Development Guidelines	Guideline	Software Development
DOC045	User Account Creation Support	Support guide	Technical Support

NexaTech Solutions: ontology and conceptual graph of organizational knowledge

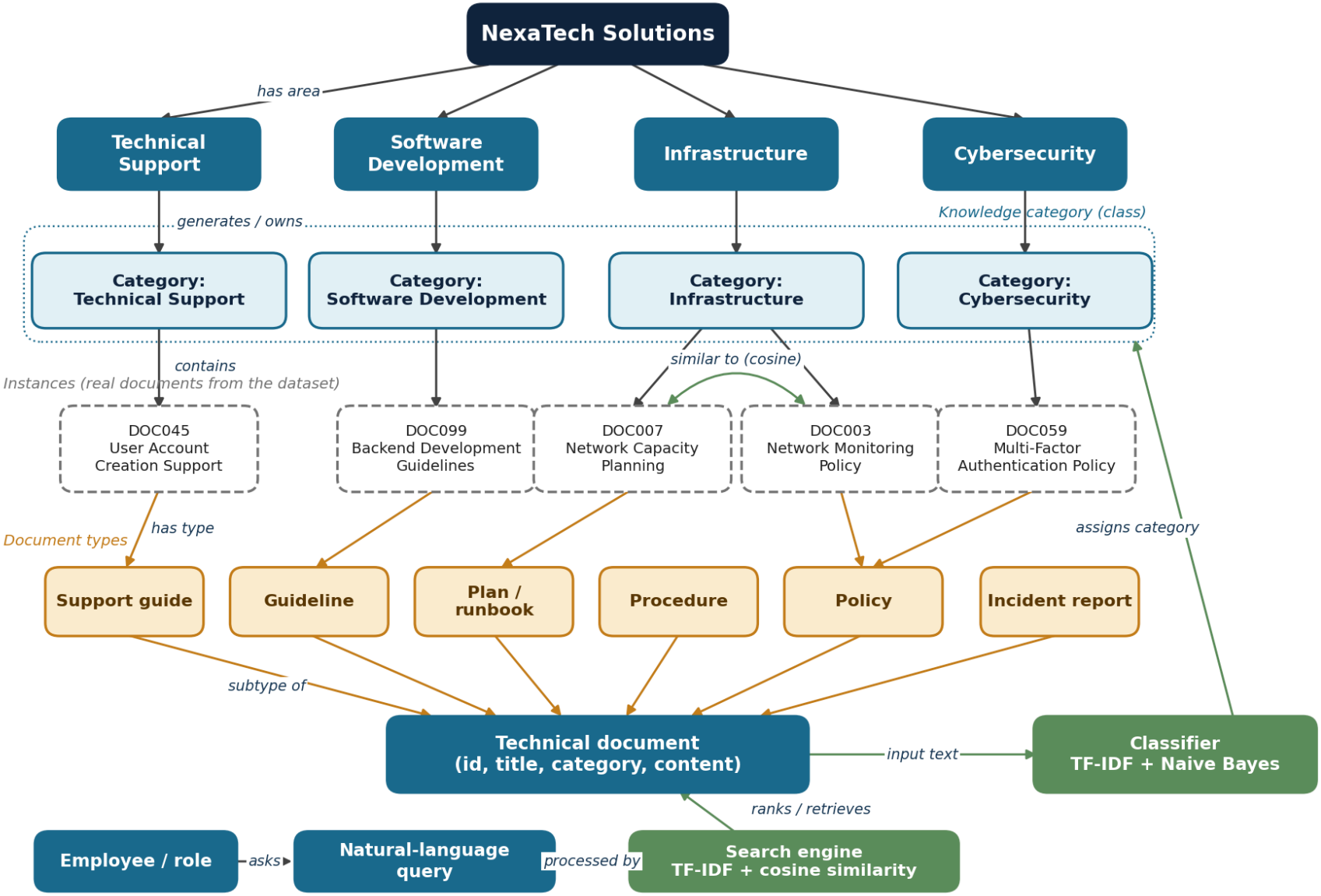


Figure 2. Conceptual graph of NexaTech: organization, areas, categories, instances, document types and models. Source: own elaboration.

4. How the prototype works

The prototype is a web application written in Python. Streamlit provides the interface, pandas handles the data and scikit-learn provides TF-IDF, cosine similarity and Naive Bayes (Pedregosa et al., 2011). PDF files are read with pypdf; scanned PDFs can be read with optional OCR (PyMuPDF and Tesseract). The scripts in SRC run the same techniques from the command line and save their results in RESULTS.

Repository structure

```
NexaTech-Knowledge-Management/
|-- app.py                          Streamlit application (interface + models)
|-- DATA/
|   |-- documents.csv               Knowledge base (102 documents)
|   `-- test_documents.csv          8 new documents for classification tests
|-- SRC/
|   |-- check_documents.py          Dataset validation and summary
|   |-- Classify_documents.py       Classification prototype in 7 stages
|   |-- evaluate_classifier.py      3- and 5-fold cross-validation, model comparison
|   `-- semantic_search.py          TF-IDF + cosine similarity search (query input)
|-- EXERCISES/                      Practical exercises, Topics 1 to 6 (+ outputs)
|-- DIAGRAMS/                       Life cycle map and conceptual graph
|-- RESULTS/                        Predictions, confusion matrices, search results
|-- DOCS/                           This report, its source and the screenshots
|-- requirements.txt
`-- README.md
```

Data flow

- **Loading:** documents.csv is read and validated (columns document_id, title, category and content; no empty values or duplicated IDs).
- **Working text:** each document gets search_text = title + content.
- **Vectorization:** TfidfVectorizer (English stop words, 1-2 grams, sublinear TF for search) turns the texts into vectors.
- **Cached models:** the models are trained once and rebuilt when the dataset changes.

Modules

Module	Function	Technique
Dashboard	Total documents, categories, average per category and chart of documents by category.	pandas, Streamlit charts
Semantic Search	Takes a natural-language query, computes its similarity with every document and shows the best results with their score. Filters: category, number of results and minimum similarity.	TF-IDF + cosine similarity
Document Classifier	Predicts the category of a new title or text and shows the score of each category.	TF-IDF + MultinomialNB (α = 0.1)
Add Document	Adds documents manually or imports CSV, PDF or TXT files; avoids duplicated IDs and contents and retrains the models.	pypdf, optional OCR
Knowledge Base	Browse and filter all stored documents, with keyword search in title or content.	pandas

How the search is computed

The query is transformed into the same TF-IDF space as the documents. The cosine similarity between the query vector *q* and each document vector *d*, cos(q, d) = (q · d) / (‖q‖ ‖d‖), is then calculated; a value closer to 1 means they share relevant terms (Manning et al., 2008). Documents with a similarity above zero are sorted from highest to lowest. The method is called semantic search in the project, but it is strictly lexical: it ranks by shared, weighted words and does not understand synonyms or paraphrases. The score measures textual similarity, not the probability that the answer is correct. Embeddings are the natural next step to capture meaning. `SRC/semantic_search.py` receives the query as an argument or in interactive mode and also prints the query terms and the terms each document matched, which explains every score.

How the classification works

The classifier learns from the documents that already have a category: TF-IDF represents each text and multinomial Naive Bayes estimates the most likely category. The score shown for each category is the model's estimated probability. The interface shows the predicted category and the score of each class, with a warning that it is an automated suggestion whose reliability depends on the amount and variety of training documents. `SRC/Classify_documents.py` runs the process in seven identifiable stages:

Stage	What the script does
1. Load the document set	Reads `DATA/documents.csv` (102 documents) and prints the category counts and the first input documents.
2. Train / test split	Stratified 75 % / 25 % split: 76 training and 26 testing documents.
3. TF-IDF vectorization	English stop words removed, unigrams and bigrams; vocabulary of 3,539 terms on the training set.
4. Train the models	Multinomial Naive Bayes with α = 0.1 (the model of the app) and Logistic Regression as a comparison.
5. Evaluate	Accuracy, precision, recall, F1, confusion matrix and the predicted category of each test document.
6. Classify new documents	Retrains on all 102 documents and classifies the 8 documents of `DATA/test_documents.csv` into the four predefined categories.
7. Save the results	`RESULTS/classification_results.csv`, `new_documents_predictions.csv` and confusion matrices.

Why α = 0.1. In the first version the classifier used the default smoothing of Naive Bayes (α = 1.0). With so few words per document, that smoothing hides the evidence of the smaller categories and the model predicted Infrastructure for 58 % of the documents, although only 32 % belong to it. Lowering α to 0.1 raised the cross-validation accuracy from 66.7 % to 84.3 % (3 folds) and the recall of Software Development in 5-fold cross-validation from 0.15 to 0.80 (section 5.6 and Topic 5).

5. Evidence of prototype development

The following screenshots show the prototype running. To reproduce them: `pip install -r requirements.txt`, then `streamlit run app.py` for the application and the commands of section 7 for the scripts.

5.1 Dashboard and input documents

The knowledge base has 102 documents in 4 categories (25.5 per category on average): Cybersecurity (29), Infrastructure (33), Software Development (20) and Technical Support (20).

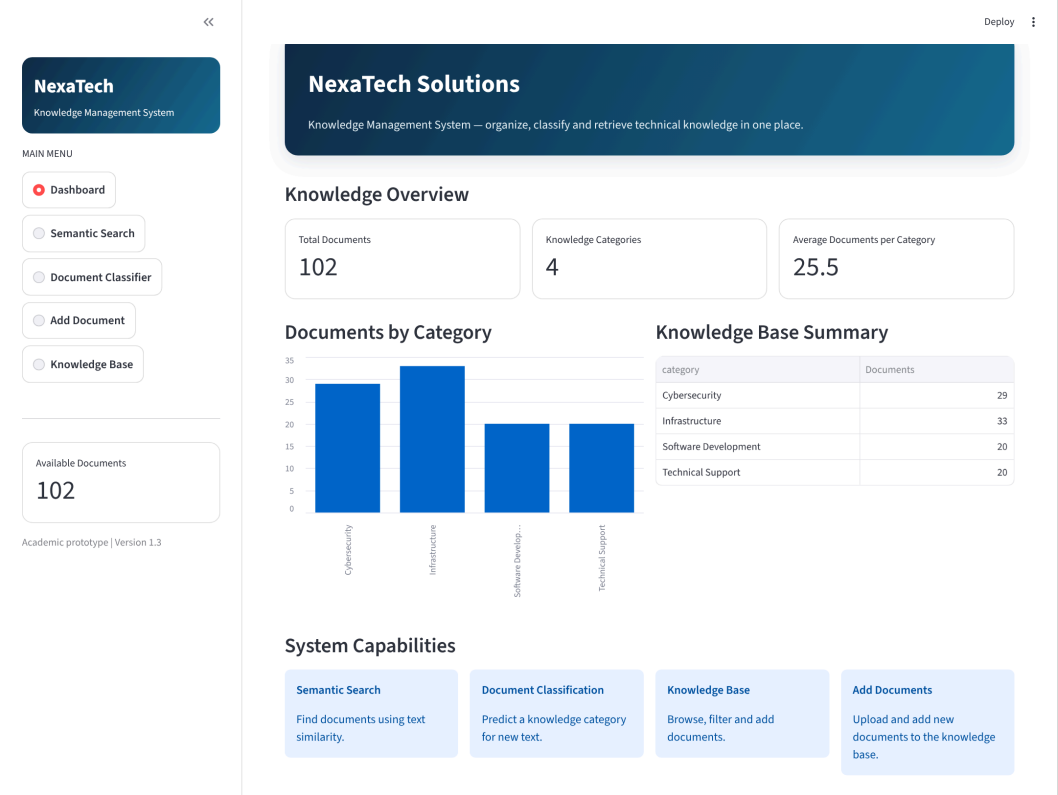


Figure 3. Dashboard with the overview of the knowledge base.

```
NexaTech-Knowledge-Management — bash

$ python SRC/check_documents.py
TEST: The Python script is running!
Dataset path: /home/claude/final/new/DATA/documents.csv
Dataset loaded successfully!
Number of documents: 102

Documents per category:
category
Infrastructure          33
Cybersecurity           29
Technical Support       20
Software Development    20
Name: count, dtype: int64

First five documents:
  document_id                          title        category
0      DOC001  Network Infrastructure Overview  Infrastructure
1      DOC002    Server Maintenance Procedures  Infrastructure
2      DOC003        Network Monitoring Policy  Infrastructure
3      DOC004    Hardware Inventory Management  Infrastructure
4      DOC005    Data Center Access Procedures  Infrastructure
```

Figure 4. Input documents loaded and validated by SRC/check_documents.py.

5.2 Classification with the repository script

```
NexaTech-Knowledge-Management — bash

$ python SRC/Classify_documents.py

======================================================================
STAGE 1 - LOAD THE DOCUMENT SET
======================================================================
File: DATA/documents.csv | Documents loaded: 102
category
Infrastructure          33
Cybersecurity           29
Technical Support       20
Software Development    20

Input documents (first 5):
document_id                          title       category
     DOC001 Network Infrastructure Overview Infrastructure
     DOC002   Server Maintenance Procedures Infrastructure
     DOC003       Network Monitoring Policy Infrastructure
     DOC004   Hardware Inventory Management Infrastructure
     DOC005   Data Center Access Procedures Infrastructure

======================================================================
STAGE 2 - TRAIN / TEST SPLIT (75% / 25%, stratified)
======================================================================
Training documents: 76 | Testing documents: 26

======================================================================
STAGE 3 - TF-IDF VECTORIZATION
======================================================================
Vocabulary size: 3539 terms (words + bigrams)
Training matrix: 76 documents x 3539 terms
Top TF-IDF terms of 'Application Requirements Documentation':
   requirements               0.284
   application requirements   0.275
   application                0.210
   traceable stakeholder      0.138
   traceable                  0.138
```

Figure 5. Stages 1 to 3 of Classify_documents.py: input documents, train/test split and TF-IDF vectorization.

```
NexaTech-Knowledge-Management — bash

$ python SRC/Classify_documents.py
   ... (earlier output omitted)
======================================================================
STAGE 4/5 - TRAIN AND EVALUATE THE CLASSIFIERS
======================================================================

>>> Naive Bayes - accuracy on the test set: 92.31%
                     precision    recall  f1-score   support

       Cybersecurity       1.00      0.86      0.92         7
      Infrastructure       0.90      1.00      0.95         9
Software Development       0.80      0.80      0.80         5
   Technical Support       1.00      1.00      1.00         5

            accuracy                           0.92        26
           macro avg       0.93      0.91      0.92        26
        weighted avg       0.93      0.92      0.92        26


>>> Logistic Regression - accuracy on the test set: 96.15%
                     precision    recall  f1-score   support

       Cybersecurity       1.00      0.86      0.92         7
      Infrastructure       1.00      1.00      1.00         9
Software Development       0.83      1.00      0.91         5
   Technical Support       1.00      1.00      1.00         5

            accuracy                           0.96        26
           macro avg       0.96      0.96      0.96        26
        weighted avg       0.97      0.96      0.96        26

Model used by the prototype (app.py): Naive Bayes (92.31%) | comparison: Logistic Regression (96.15%)
```

Figure 6. Stages 4 and 5: training and evaluation of Naive Bayes and Logistic Regression on 26 test documents.

```
NexaTech-Knowledge-Management — bash

$ python SRC/Classify_documents.py
   ... (earlier output omitted)
Model used by the prototype (app.py): Naive Bayes (92.31%) | comparison: Logistic Regression (96.15%)

Predicted category for each test document:
  [OK ] DOC012       Virtual Machine Management           actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC016       Storage Infrastructure Management    actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC050       User Onboarding Technical Support    actual=Technical Support     predicted=Technical Support
  [OK ] DOC039       Infrastructure Resource Allocation   actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC097       Release Version Management           actual=Software Development  predicted=Software Development
  [OK ] DOC061       Access Review and Authorization      actual=Cybersecurity         predicted=Cybersecurity
  [OK ] DOC060       Data Privacy and Handling Policy     actual=Cybersecurity         predicted=Cybersecurity
  [OK ] DOC034       Server Room Environmental Controls   actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC094       Unit Testing Guidelines              actual=Software Development  predicted=Software Development
  [OK ] DOC064       Security Risk Assessment             actual=Cybersecurity         predicted=Cybersecurity
  [OK ] DOC084       Software Testing Strategy            actual=Software Development  predicted=Software Development
  [OK ] DOC042       Email Configuration Support          actual=Technical Support     predicted=Technical Support
  [OK ] DOC026       Network Performance Optimization     actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC023       Software Installation Support        actual=Technical Support     predicted=Technical Support
  [ERR] DOC095       Application Performance Optimization actual=Software Development  predicted=Infrastructure
  [ERR] DOC074       Secure Software Development Policy   actual=Cybersecurity         predicted=Software Development
  [OK ] DOC029       Infrastructure Asset Lifecycle       actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC015       Network Cabling Standards            actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC049       Blue Screen Error Resolution         actual=Technical Support     predicted=Technical Support
  [OK ] DOC072       Vulnerability Management Process     actual=Cybersecurity         predicted=Cybersecurity
  [OK ] DOC073       Removable Media Security             actual=Cybersecurity         predicted=Cybersecurity
  [OK ] DOC019       Infrastructure Incident Response     actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC083       Version Control Management           actual=Software Development  predicted=Software Development
  [OK ] DOC048       Application Performance Support      actual=Technical Support     predicted=Technical Support
  [OK ] DOC036       Database Server Infrastructure       actual=Infrastructure        predicted=Infrastructure
  [OK ] DOC068       Privileged Account Management        actual=Cybersecurity         predicted=Cybersecurity
```

Figure 7. Predicted category of each test document (24 of 26 correct).

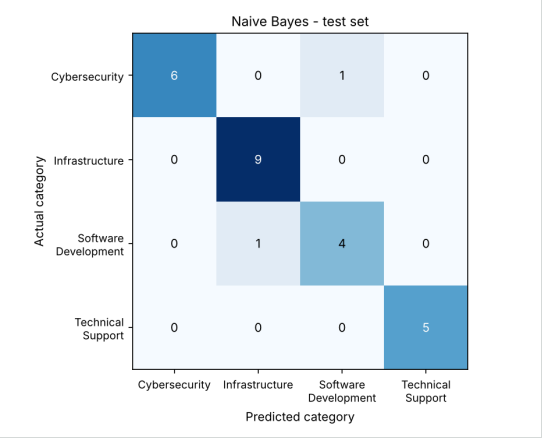


Figure 8. Confusion matrix of Naive Bayes on the test set.

Analysis: Naive Bayes reaches 92.3 % accuracy on the 26 test documents (Logistic Regression 96.2 %). Technical Support and Infrastructure are fully recalled. The two errors are borderline documents: *Application Performance Optimization* (DOC095, Software Development) shares performance and monitoring vocabulary with Infrastructure, and *Secure Software Development Policy* (DOC074, Cybersecurity) is about software development by definition. Software Development recall rose from 0.20 in the first version to 0.80.

5.3 Classification tests with new documents

Eight documents that are not in the training data (`DATA/test_documents.csv`, two per category) were classified by the script. The first three are the tests of the first version of this report.

```
NexaTech-Knowledge-Management — bash

$ python SRC/Classify_documents.py
   ... (earlier output omitted)

======================================================================
STAGE 6 - CLASSIFY NEW DOCUMENTS (DATA/test_documents.csv)
======================================================================
  [OK ] TEST001  Endpoint Protection and Incident Respons -> Cybersecurity         (score 0.87) expected: Cybersecurity
  [OK ] TEST002  API Versioning and Documentation Standar -> Software Development  (score 0.84) expected: Software Development
  [OK ] TEST003  Vulnerability Assessment and Patch Manag -> Cybersecurity         (score 0.87) expected: Cybersecurity
  [OK ] TEST004  Load Balancer Configuration Procedure    -> Infrastructure        (score 0.95) expected: Infrastructure
  [OK ] TEST005  Data Center Cooling Maintenance          -> Infrastructure        (score 0.97) expected: Infrastructure
  [OK ] TEST006  Printer Driver Installation Support      -> Technical Support     (score 0.95) expected: Technical Support
  [OK ] TEST007  New Laptop Setup for Employees           -> Technical Support     (score 0.85) expected: Technical Support
  [OK ] TEST008  Code Refactoring Guidelines              -> Software Development  (score 0.90) expected: Software Development

Correct: 8 of 8 new documents

NEW DOCUMENT TEST
Document: Employees must enable multi-factor authentication, protect passwords, and report suspicious emails.
Predicted category: Cybersecurity

======================================================================
STAGE 7 - SAVE RESULTS
======================================================================
RESULTS/classification_results.csv       (test-set predictions)
RESULTS/new_documents_predictions.csv    (new documents)
RESULTS/confusion_matrix_*.png           (confusion matrices)
```

Figure 9. Stage 6: the eight new documents are assigned to the predefined categories (8/8 correct).

Test text	Expected category	Predicted category	Score	Correct?
Endpoint Protection and Incident Response	Cybersecurity	Cybersecurity	87.2%	**Yes**
API Versioning and Documentation Standards	Software Development	Software Development	84.3%	**Yes**
Vulnerability Assessment and Patch Management	Cybersecurity	Cybersecurity	86.6%	**Yes**
Load Balancer Configuration Procedure	Infrastructure	Infrastructure	95.5%	**Yes**
Data Center Cooling Maintenance	Infrastructure	Infrastructure	96.9%	**Yes**
Printer Driver Installation Support	Technical Support	Technical Support	95.4%	**Yes**
New Laptop Setup for Employees	Technical Support	Technical Support	84.7%	**Yes**
Code Refactoring Guidelines	Software Development	Software Development	90.4%	**Yes**

The same three tests of the first version were repeated in the application (Figures 10 to 12). In the first version the API document was classified as Infrastructure (33.5 %); with α = 0.1 it is classified correctly as Software Development (84.3 %), and the scores of the other two tests rose from 46.0 % and 39.9 % to 87.2 % and 86.6 %, which means higher confidence.

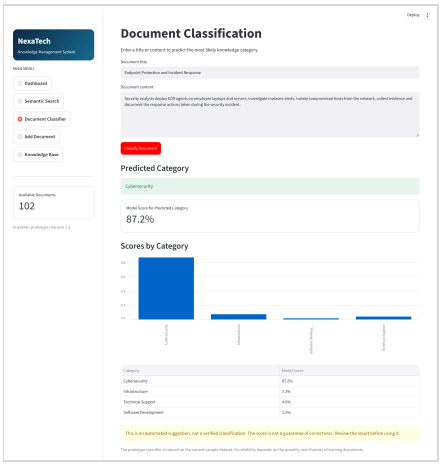


Figure 10. Test 1: Endpoint Protection and Incident Response → Cybersecurity (87.2%).

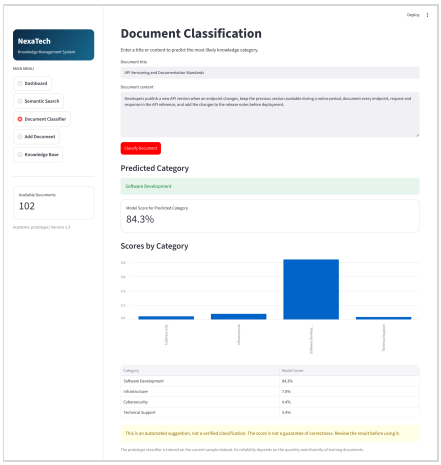


Figure 11. Test 2: API Versioning and Documentation Standards → Software Development (84.3%).

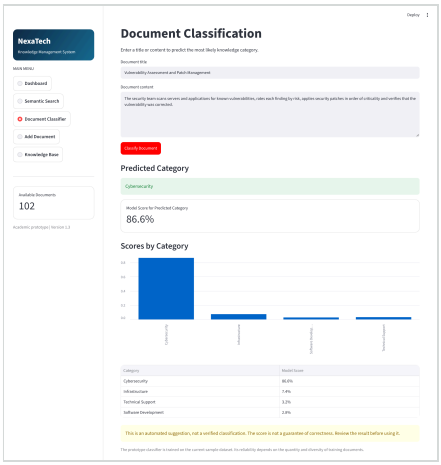


Figure 12. Test 3: Vulnerability Assessment and Patch Management → Cybersecurity (86.6%).

```
NexaTech-Knowledge-Management — bash

$ python SRC/Classify_documents.py --text "my laptop screen is broken and I need help installing the VPN client"
Text: my laptop screen is broken and I need help installing the VPN client
Predicted category: Technical Support

  Technical Support      0.513 ####################
  Cybersecurity          0.241 #########
  Software Development   0.152 ######
  Infrastructure         0.094 ###
```

Figure 13. Classification of a free text written by the team: Technical Support (51.3%), Cybersecurity second because of the word "VPN".

5.4 Semantic search examples

Example 1. Query: *how can we protect employee acounts* (typed with a spelling mistake, exactly as it was run).

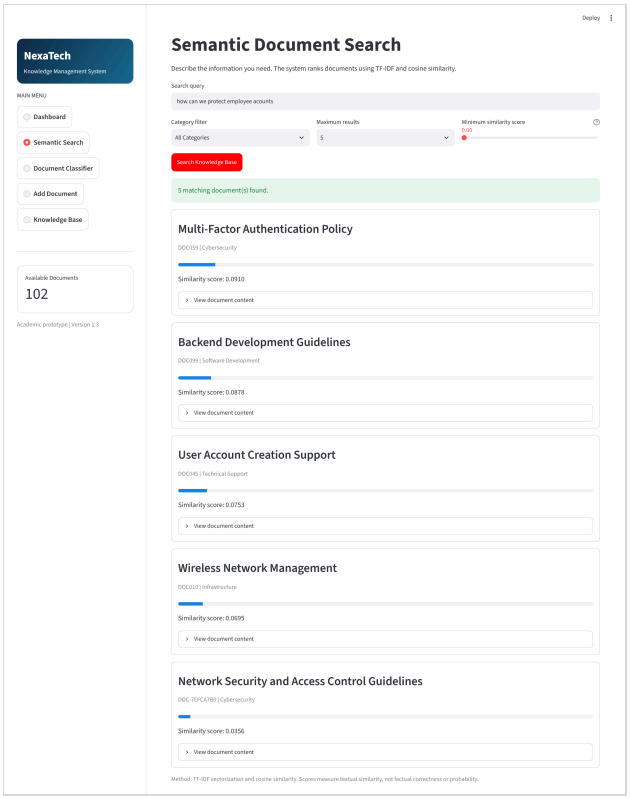


Figure 14. Search results for protecting employee accounts (application screenshot).

#	Retrieved document	Category	Cosine similarity
1	DOC059 · Multi-Factor Authentication Policy	Cybersecurity	0.0910
2	DOC099 · Backend Development Guidelines	Software Development	0.0878
3	DOC045 · User Account Creation Support	Technical Support	0.0753

Interpretation: the multi-factor authentication policy is the most relevant result for protecting accounts. The second result (backend guidelines) only shares the general term “protect” and is not relevant, and it ranks above account-creation support, which is more related to the topic. Scores are low because the query is short and contains a misspelled word (“acounts”), which removes an exact term match in TF-IDF: only “protect” and “employee” remain in the query vector. Example 3 repeats the question without the mistake.

Example 2. Query: *What are the procedures for handling network failures or capacity issues?*

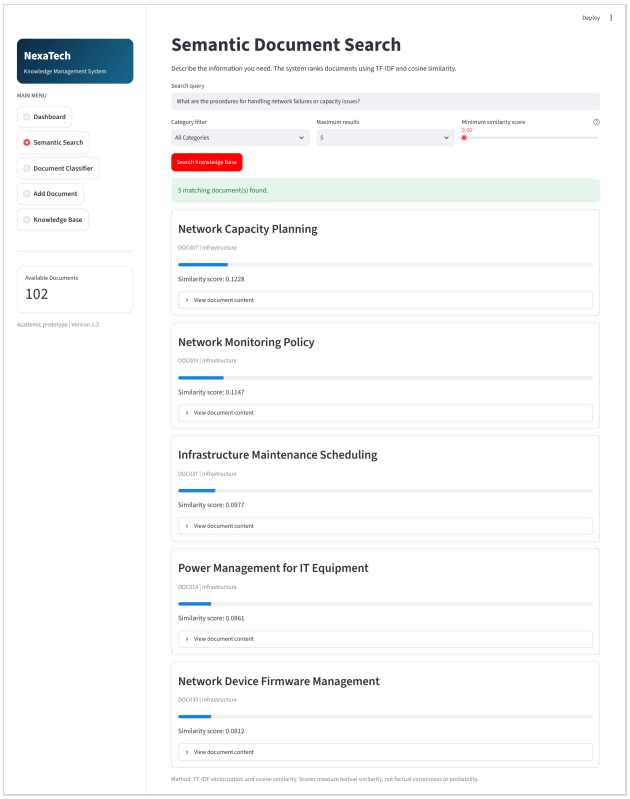


Figure 15. Search results for network failures and capacity issues (application screenshot).

#	Retrieved document	Category	Cosine similarity
1	DOC007 · Network Capacity Planning	Infrastructure	0.1228
2	DOC003 · Network Monitoring Policy	Infrastructure	0.1147
3	DOC037 · Infrastructure Maintenance Scheduling	Infrastructure	0.0977

Interpretation: the three top documents belong to Infrastructure and deal with capacity, monitoring and maintenance of the network, so they answer the information need directly. DOC007 matches “capacity” and “network”, DOC003 also “failures”. Scores are higher than in example 1 because the query is more specific and uses terms that appear in those documents.

Examples 3 and 4 (script). `SRC/semantic_search.py` prints the query terms kept after removing stop words, the score of each document and the terms it matched.

```
NexaTech-Knowledge-Management — bash

$ python SRC/semantic_search.py --top 3
NEXATECH SEMANTIC DOCUMENT SEARCH
==========================================
Indexed documents: 102 | Vocabulary: 4435 terms

Query: how can we protect employee acounts
Query vector: 2 TF-IDF terms -> employee, protect
document_id                          title            category similarity_score matched_terms
     DOC059 Multi-Factor Authentication Policy        Cybersecurity          0.0910       protect
     DOC099     Backend Development Guidelines Software Development          0.0878       protect
     DOC045      User Account Creation Support    Technical Support          0.0753      employee

Query: What are the procedures for handling network failures or capacity issues?
Query vector: 6 TF-IDF terms -> capacity, failures, handling, issues, network, procedures
document_id                              title       category similarity_score              matched_terms
     DOC007            Network Capacity Planning Infrastructure          0.1228           capacity, network
     DOC003            Network Monitoring Policy Infrastructure          0.1147 capacity, failures, network
     DOC037 Infrastructure Maintenance Scheduling Infrastructure          0.0977            failures, issues
```

Figure 16. Examples 1 and 2 reproduced with the script, with the matched terms of each document.

```
NexaTech-Knowledge-Management — bash

$ python SRC/semantic_search.py --top 3
   ... (earlier output omitted)

Query: How can we protect employee accounts and passwords?
Query vector: 4 TF-IDF terms -> accounts, employee, passwords, protect
document_id                            title          category similarity_score       matched_terms
     DOC059 Multi-Factor Authentication Policy     Cybersecurity          0.1328   accounts, protect
     DOC071         Secure Password Management     Cybersecurity          0.1119 accounts, passwords
     DOC045      User Account Creation Support Technical Support          0.0972  accounts, employee

Query: How do developers test and fix software bugs?
Query vector: 4 TF-IDF terms -> developers, developers test, software, test
document_id                         title             category similarity_score                    matched_terms
     DOC094        Unit Testing Guidelines Software Development          0.1921 developers, developers test, test
     DOC084      Software Testing Strategy Software Development          0.1191                   software, test
     DOC081 Software Development Lifecycle Software Development          0.0814                   software, test

Results saved to RESULTS/semantic_search_results.csv
```

Figure 17. Examples 3 and 4 run with the script.

Query	Top results (cosine similarity)	Why they answer the information need
3. How can we protect employee accounts and passwords?	DOC059 Multi-Factor Authentication Policy (0.1328) DOC071 Secure Password Management (0.1119) DOC045 User Account Creation Support (0.0972)	With correct spelling the query keeps “accounts” and “passwords”: the scores rise and the irrelevant backend guideline of example 1 is replaced by the password management document.
4. How do developers test and fix software bugs?	DOC094 Unit Testing Guidelines (0.1921) DOC084 Software Testing Strategy (0.1191) DOC081 Software Development Lifecycle (0.0814)	All three are Software Development documents about testing; the bigram “developers test” gives DOC094 the highest score.

Examples 5 and 6 (interactive mode). The user types questions in natural language one after another. Example 5 is the VPN case of section 2.

```
NexaTech-Knowledge-Management — bash

$ python SRC/semantic_search.py --interactive --top 3
NEXATECH SEMANTIC DOCUMENT SEARCH
==========================================
Indexed documents: 102 | Vocabulary: 4435 terms
Type a question (press Enter on an empty line to exit).

search> the VPN connection drops after the firmware update

Query: the VPN connection drops after the firmware update
Query vector: 4 TF-IDF terms -> connection, firmware, update, vpn
document_id                                title          category similarity_score    matched_terms
     DOC046        VPN Connectivity Troubleshooting Technical Support           0.1406              vpn
     DOC030      Network Device Firmware Management    Infrastructure           0.1124         firmware
     DOC017 Operating System Infrastructure Updates    Infrastructure           0.1119 firmware, update

search> how do we review code before merging

Query: how do we review code before merging
Query vector: 3 TF-IDF terms -> code, merging, review
document_id                        title             category similarity_score matched_terms
     DOC083   Version Control Management Software Development           0.1535 code, merging
     DOC089        Code Review Procedure Software Development           0.1305  code, review
     DOC080 Application Security Testing        Cybersecurity           0.0776  code, review

search>
```

Figure 18. Interactive search: the VPN case returns VPN Connectivity Troubleshooting first (0.1406); the code review question returns Version Control Management (0.1535) and Code Review Procedure (0.1305).

Interpretation: for the VPN case the first result is the support guide for VPN connectivity, and the next two are about firmware and updates, the cause of the problem. For the code review question, the first two results are the version control (branches, merging) and code review documents; the third, Application Security Testing, shares “code” and “review” but is less useful.

5.5 Knowledge base browsing

The Knowledge Base module filters by category and searches by keyword in title or content. It shows all stored documents with their ID, title, category and content.

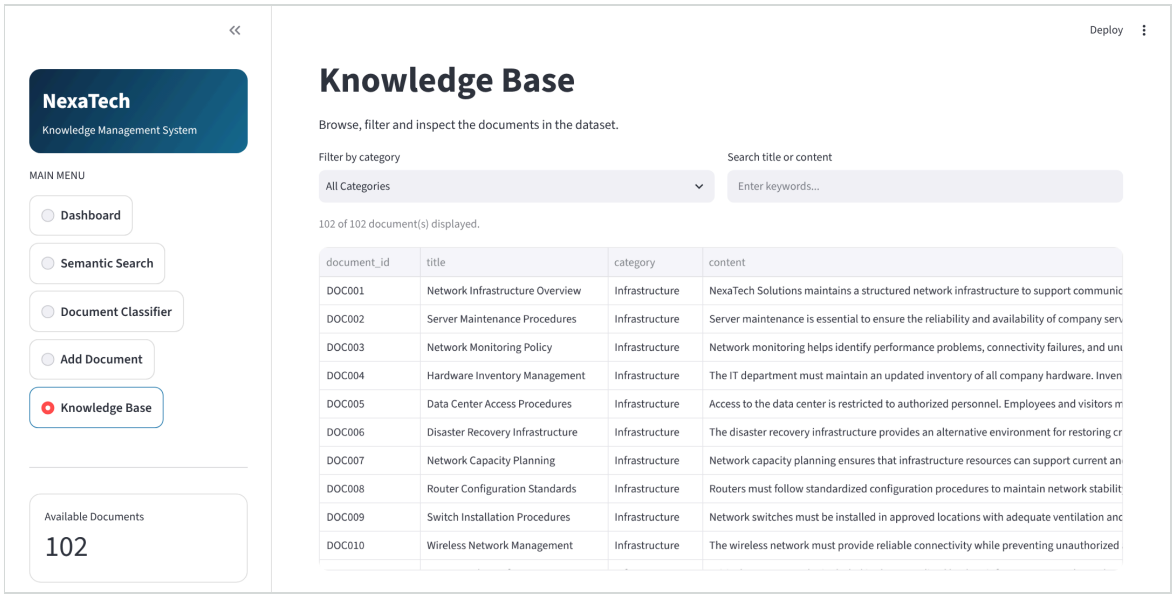


Figure 19. Knowledge Base module with the document set.

5.6 Classifier evaluation with the repository scripts

Two scripts in SRC measure the classifier on the committed dataset (102 documents). The hold-out predictions are saved in `RESULTS/classification_results.csv`. The table compares the first version of the classifier with the current one.

Configuration	Script	Hold-out 75/25	3-fold CV	5-fold CV
First version: Naive Bayes, α = 1.0	Classify_documents.py (v1)	76.9%	66.7%	72.4%
Current: Naive Bayes, α = 0.1 (app.py)	Classify_documents.py	**92.3%**	**84.3%**	**85.1%**
Comparison: Logistic Regression (balanced)	evaluate_classifier.py	96.2%	90.2%	89.1%

NexaTech-Knowledge-Management — bash

```
$ python SRC/evaluate_classifier.py
CROSS-VALIDATION RESULTS
========================================================================
Total documents: 102
Categories: 4  Infrastructure: 33, Cybersecurity: 29, Technical Support: 20, Software Development: 20

3-fold stratified cross-validation
  Model                                  Mean     Std   Folds
  NB alpha=1.0, unigrams (original SRC)     66.67%   5.00%   0.735, 0.618, 0.647
  NB alpha=1.0, uni+bigrams (orig. app)     59.80%   5.00%   0.647, 0.529, 0.618
  NB alpha=0.1, uni+bigrams (app.py)        84.31%   5.55%   0.882, 0.765, 0.882
  Logistic Regression (balanced)            90.20%   1.39%   0.912, 0.882, 0.912

5-fold stratified cross-validation
  Model                                  Mean     Std   Folds
  NB alpha=1.0, unigrams (original SRC)     72.38%   9.41%   0.905, 0.714, 0.650, 0.650, 0.700
  NB alpha=1.0, uni+bigrams (orig. app)     66.52%   8.31%   0.810, 0.667, 0.650, 0.550, 0.650
  NB alpha=0.1, uni+bigrams (app.py)        85.14%   7.21%   0.952, 0.905, 0.750, 0.850, 0.800
  Logistic Regression (balanced)            89.14%   3.85%   0.905, 0.952, 0.850, 0.850, 0.900

Interpretation: with alpha=1.0 the smoothing is so strong that the word
evidence of the small categories (20 documents) is diluted and the model
favours Infrastructure (33 documents). alpha=0.1 fixes most of these errors.
```

Figure 20. Cross-validation with SRC/evaluate_classifier.py (original and current configurations).

Analysis: the first version reached 76.9 % on the hold-out split but only 66.7 % in cross-validation, and Software Development was missed 4 times out of 5. The cause was not only the size of the dataset: the default smoothing α = 1.0 pushed the model towards the largest category (Infrastructure). With α = 0.1 every configuration improves, and the cross-validation results (84–85 %) are now close to the hold-out result, so the estimate is more stable. Logistic Regression is a little better, but Naive Bayes was kept because it is simple, fast to retrain in the app and gives interpretable probabilities. A realistic expectation for the current dataset is between 84 % and 92 %.

6. Evidence of practical exercises (Topics 1 to 6)

During the first weeks of the course the team carried out practical exercises with the NexaTech documents. Each one contributed to a part of the project. The scripts are in `EXERCISES` and their outputs in `EXERCISES/outputs`; `SRC/check_documents.py` (data exploration) and `SRC/evaluate_classifier.py` (model validation) also started as exercises.

Topic	Exercise	Main result	Contribution to the project
1	Knowledge audit: tacit vs explicit knowledge, SECI model	12 knowledge assets of 4 areas, half of them tacit; how many documents already capture each one	Defined the four knowledge categories and the tacit/explicit rows of the life cycle map
2	Text preprocessing, Bag-of-Words and TF-IDF by hand	“authentication” (in 1 of 3 docs) gets idf 1.69, “accounts” (2 of 3) gets 1.29	Justified stop-word removal and TF-IDF for search and classification
3	Knowledge representation: the ontology as triples	325 triples (19 schema, 306 instance); document types assigned to all 102 documents	Validated the ontology of Figure 2 with real instances
4	Information retrieval: Boolean model vs vector space model	Boolean AND returns 0–1 documents per query; TF-IDF ranking MAP = 0.699	Chose ranked TF-IDF + cosine similarity for the search
5	Classification tests: features, models and smoothing	CV 0.84–0.93; α = 1.0 predicts Infrastructure for 58 % of documents; α = 0.1 reaches 0.85	Found the cause of the low accuracy and changed α to 0.1 in app.py
6	Document similarity and K-Means clustering	Same-category similarity 2.1× higher; purity 0.58 with 4 clusters	Supports the “is similar to” relation and shows that categories share vocabulary

Topic 1 – Knowledge audit

NexaTech-Knowledge-Management — bash

```
$ python EXERCISES/topic1_knowledge_audit.py
Team                  Knowledge asset                          Type      SECI             Docs found
-------------------------------------------------------------------------------------------------
IT Infrastructure     Server maintenance steps                 explicit  Internalization           8
IT Infrastructure     Experience recovering from incidents     tacit     Externalization           4
IT Infrastructure     Network configuration standards          explicit  Combination              18
Help Desk             How senior technicians diagnose problems tacit     Socialization             2
Help Desk             Solutions to frequent user problems      tacit     Externalization           6
Help Desk             Ticket priority rules                    explicit  Internalization           1
Cybersecurity         Security policies (passwords, encryption) explicit Combination              6
Cybersecurity         Intuition to detect attacks              tacit     Socialization             6
Cybersecurity         Lessons from security incidents          tacit     Externalization           6
Software Development  Coding and review standards              explicit  Combination               9
Software Development  Debugging know-how of senior developers  tacit     Socialization             4
Software Development  Deployment steps                         explicit  Internalization           3

Tacit vs explicit: {'explicit': 6, 'tacit': 6}
SECI conversions:  {'Internalization': 3, 'Externalization': 3, 'Combination': 3, 'Socialization': 3}
Knowledge categories needed: ['Cybersecurity', 'Infrastructure', 'Software Development', 'Technical Support']

Conclusion: half of the critical knowledge is tacit (diagnosis experience,
lessons from incidents). The KMS must externalize it as documents and make
it easy to classify and find.
```

Figure 21. Knowledge assets per area, type of knowledge, SECI conversion and documents found in the dataset.

Topic 2 – Text preprocessing and TF-IDF

A sentence of DOC059 is followed through tokenization, stop-word removal and stemming; then a Bag-of-Words matrix and TF-IDF weights are computed by hand for three password-related documents and compared with scikit-learn.

NexaTech-Knowledge-Management — bash

```
$ python EXERCISES/topic2_text_preprocessing.py
Original sentence (DOC059 Multi-Factor Authentication Policy):
  Multi-factor authentication adds an additional verification step to protect company accounts.

1) Tokens (12): ['Multi', 'factor', 'authentication', 'adds', 'an', 'additional', 'verification', 'step', 'to', 'protect', 'company', 'accounts']
2) Without stop words (10): ['multi', 'factor', 'authentication', 'adds', 'additional', 'verification', 'step', 'protect', 'company', 'accounts']
3) Stemmed: ['multi', 'factor', 'authentic', 'add', 'additional', 'verific', 'step', 'protect', 'company', 'account']

4) Bag-of-Words for ['DOC021', 'DOC059', 'DOC071']: matrix (3, 73)
     count('password') = [2, 0, 2]
     count('passwords') = [1, 0, 2]
     count('accounts') = [0, 2, 1]
     count('authentication') = [0, 3, 0]
     count('employees') = [0, 1, 1]

5) TF-IDF by hand  (idf = ln((1+N)/(1+df)) + 1, N=3) for document DOC059:
     accounts       tf(DOC059)=2 df=2 idf=1.288 tf-idf=2.575
     password       tf(DOC059)=0 df=2 idf=1.288 tf-idf=0.000
     authentication  tf(DOC059)=3 df=1 idf=1.693 tf-idf=5.079
     mfa            tf(DOC059)=1 df=1 idf=1.693 tf-idf=1.693
   -> terms present in fewer documents get a higher idf (more discriminative).

6) scikit-learn TF-IDF (L2-normalized), top terms of DOC059:
     authentication 0.512
     accounts       0.259
     reported       0.171
     mfa            0.171
     notifications  0.171
     step           0.171
```

Figure 22. Topic 2 output.

Topic 3 – Knowledge representation

The conceptual graph of Figure 2 is encoded as subject–predicate–object triples and populated with the dataset: every document gets its category, its document type (from the title) and its most similar document.

```
NexaTech-Knowledge-Management — bash

$ python EXERCISES/topic3_knowledge_representation.py
Knowledge graph: 325 triples (19 schema-level, 306 instance-level)

Q1. Document types per knowledge category (instances of Figure 2):
    Technical Support      Support guide: 13, Procedure: 4, Policy: 2, Plan / runbook: 1
    Software Development   Guideline: 12, Plan / runbook: 5, Procedure: 3
    Infrastructure         Guideline: 14, Plan / runbook: 13, Procedure: 3, Policy: 2, Incident report: 1
    Cybersecurity          Guideline: 12, Policy: 6, Plan / runbook: 6, Incident report: 3, Procedure: 2

Q2. Which policies exist in Cybersecurity?
    DOC055        Phishing Awareness Policy
    DOC059        Multi-Factor Authentication Policy
    DOC060        Data Privacy and Handling Policy
    DOC063        Secure Remote Work Policy
    DOC074        Secure Software Development Policy
    DOC079        Secure Network Access Policy

Q3. Instances of Figure 2 and their most similar document (relation 'similar to'):
    DOC007 -> DOC016  cosine=0.233  (Storage Infrastructure Management)
    DOC003 -> DOC033  cosine=0.329  (Infrastructure Monitoring Alerts)
    DOC059 -> DOC-7EFCA7B0  cosine=0.256  (Network Security and Access Control Guidelines)
    DOC045 -> DOC075  cosine=0.271  (Identity and Access Management)
    DOC099 -> DOC060  cosine=0.150  (Data Privacy and Handling Policy)

Q4. Path from an employee to a document:
    Employee -asks-> Natural-language query -processed by-> Search engine -ranks / retrieves-> Technical document -has type-> Policy
```

Figure 23. Topic 3 output: document types per category, queries over the graph and “is similar to” links of the instances of Figure 2.

Topic 4 – Information retrieval

An inverted index is built and four queries with manual relevance judgments are run with the Boolean model and with TF-IDF ranking, measuring Precision@3, Recall@3 and Average Precision (Manning et al., 2008).

```
NexaTech-Knowledge-Management — bash

$ python EXERCISES/topic4_information_retrieval.py
Inverted index: 1128 terms over 102 documents
  postings('backup')   = ['DOC002', 'DOC008', 'DOC011', 'DOC036', 'DOC051', 'DOC077']
  postings('phishing') = ['DOC-7EFCA7B0', 'DOC-EF5061B2', 'DOC055', 'DOC066']

Query                                        Boolean AND    P@3    R@3     AP
--------------------------------------------------------------------------------
password and account security for employees              1   0.67   0.40   0.50
    Boolean AND: ['DOC-EF5061B2'] | TF-IDF top-3: ['DOC021', 'DOC071', 'DOC066']
server backup and data recovery                          1   1.00   0.75   0.92
    Boolean AND: ['DOC011'] | TF-IDF top-3: ['DOC051', 'DOC011', 'DOC077']
software testing and bug fixing                          0   0.67   0.67   0.67
    Boolean AND: EMPTY | TF-IDF top-3: ['DOC084', 'DOC092', 'DOC080']
phishing malware security incident                       1   1.00   0.60   0.71
    Boolean AND: ['DOC-EF5061B2'] | TF-IDF top-3: ['DOC056', 'DOC058', 'DOC055']

Mean Average Precision (TF-IDF): 0.699
Boolean AND needs every word in the document, so natural-language queries
often return nothing; ranked TF-IDF still finds the relevant documents.
```

Figure 24. Topic 4 output.

Topic 5 – Classification tests

Three feature representations and three models are compared with 5-fold cross-validation; then the smoothing parameter of Naive Bayes is tested and the model is tried with short informal texts.

```
NexaTech-Knowledge-Management — bash

$ python EXERCISES/topic5_classification_tests.py
Test 1 - 5-fold cross-validation accuracy (102 documents)
Features               Naive Bayes a=0.1     Logistic Regr.         Linear SVM
------------------------------------------------------------------------------
BoW unigrams                       0.891              0.931              0.912
TF-IDF unigrams                    0.841              0.901              0.901
TF-IDF uni+bigrams                 0.851              0.891              0.891

Test 2 - Naive Bayes smoothing (TF-IDF uni+bigrams, 5-fold CV)
   alpha  accuracy   recall per category (Cyber / Infra / SoftDev / Support)
     1.0     0.667   0.86 / 0.97 / 0.15 / 0.40   (predicted Infrastructure: 58% of docs, real: 32%)
     0.5     0.755   0.83 / 0.97 / 0.45 / 0.60   (predicted Infrastructure: 48% of docs, real: 32%)
     0.3     0.824   0.86 / 0.97 / 0.65 / 0.70   (predicted Infrastructure: 42% of docs, real: 32%)
     0.1     0.853   0.86 / 0.94 / 0.80 / 0.75   (predicted Infrastructure: 39% of docs, real: 32%)
    0.05     0.853   0.83 / 0.94 / 0.80 / 0.80   (predicted Infrastructure: 37% of docs, real: 32%)

Test 3 - short informal texts typed by employees (TF-IDF + NB alpha=0.1)
  Technical Support      <- "my laptop does not turn on and the printer is not working"
  Cybersecurity          <- "someone tried to log in to my account from another country"
  Software Development   <- "the deployment pipeline failed after the last commit"
  Infrastructure         <- "the server room temperature is too high"
  Cybersecurity          <- "employees must not share confidential files"
```

Figure 25. Topic 5 output: with α = 1.0 only 15 % of Software Development documents are recognised; with α = 0.1, 80 %.

Topic 6 – Similarity and clustering

The cosine similarity between all documents is computed to recommend related documents, and K-Means groups the documents without labels to compare the clusters with the four categories.

```
NexaTech-Knowledge-Management — bash

$ python EXERCISES/topic6_similarity_and_clustering.py
Related to DOC007 (Network Capacity Planning):
    0.211  DOC016        Storage Infrastructure Management
    0.199  DOC003        Network Monitoring Policy
    0.171  DOC026        Network Performance Optimization
Related to DOC059 (Multi-Factor Authentication Policy):
    0.246  DOC-7EFCA7B0  Network Security and Access Control Guidelines
    0.215  DOC063        Secure Remote Work Policy
    0.157  DOC061        Access Review and Authorization
Related to DOC094 (Unit Testing Guidelines):
    0.136  DOC087        API Development Guidelines
    0.133  DOC084        Software Testing Strategy
    0.121  DOC038        Network Connectivity Testing

Mean similarity, same category:      0.072
Mean similarity, different category: 0.034

K-Means clusters (k=4, no labels used):
  cluster 0: 22 docs | majority = Infrastructure       | infrastructure, maintenance, administrators, recovery, server
  cluster 1: 32 docs | majority = Technical Support    | software, support, user, technicians, troubleshooting
  cluster 2: 28 docs | majority = Cybersecurity        | security, access, management, network, review
  cluster 3: 20 docs | majority = Cybersecurity        | access, employees, accounts, reported, data
Purity: 0.578 | Adjusted Rand Index: 0.214

Heatmap saved: EXERCISES/outputs/topic6_similarity_heatmap.png
```

Figure 26. Topic 6 output.

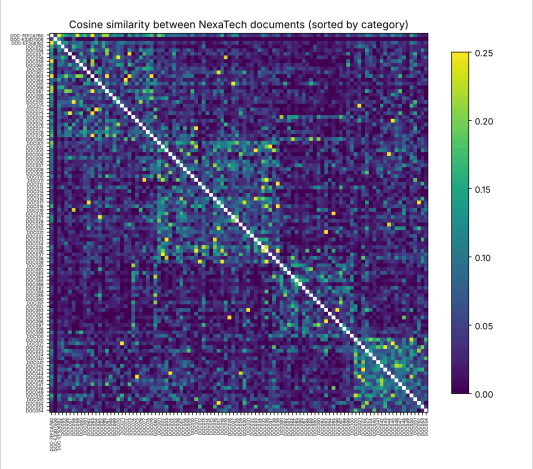


Figure 27. Cosine similarity between documents, sorted by category.

Interpretation: documents are on average 2.1 times more similar to documents of their own category, which supports the “is similar to” relation of the ontology. The clusters only partly match the categories (purity 0.58): Cybersecurity documents appear in two clusters and Technical Support shares many terms with the other areas, which is the same overlap seen in the classification errors.

7. Repository and reproduction

Repository: https://github.com/Andres145876/NexaTech-Knowledge-Management

It contains the application (app.py), the dataset and the test documents (DATA), the classification and search scripts (SRC), the practical exercises (EXERCISES), the diagrams (DIAGRAMS), the execution results (RESULTS), this report with its screenshots (DOCS) and the dependencies (requirements.txt).

```
pip install -r requirements.txt
streamlit run app.py                                  # web application

python SRC/check_documents.py                         # dataset summary
python SRC/Classify_documents.py                      # 7-stage classification + new documents
python SRC/Classify_documents.py --text "my VPN does not connect"
python SRC/evaluate_classifier.py                     # cross-validation
python SRC/semantic_search.py                         # example queries
python SRC/semantic_search.py "how can we protect employee accounts"
python SRC/semantic_search.py --interactive           # type your own queries
python EXERCISES/topic1_knowledge_audit.py            # ... up to topic6
```

8. Conclusion

The prototype puts the knowledge life cycle of NexaTech into practice: documents are added, stored, classified automatically, shared, retrieved with similarity search and updated, and every stage of the map has a component of the application behind it. The ontology describes the areas, categories, document types and models, and the exercises populated it with the 102 real documents.

The tests show that the classifier and the search work on the sample dataset and also expose their limits. The evaluation found that the low accuracy of the first version (66.7 % in cross-validation) came mainly from the default smoothing of Naive Bayes; with $\alpha = 0.1$ the classifier reaches 92.3 % on the hold-out split and 85.1 % in 5-fold cross-validation, and it classifies the 8 new test documents correctly, including the API document that failed before. Search returns the right documents for specific, well-written queries and loses quality with misspelled or very short ones, because TF-IDF compares words, not meaning.

Future work includes adding more documents to balance the categories, evaluating Logistic Regression in the app, using embeddings for semantic search, adding spelling correction to queries and adding document versioning. AI supports the process, but specialists keep the final decision.

References

Dalkir, K. (2017). *Knowledge management in theory and practice* (3rd ed.). MIT Press.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Nonaka, I., & Takeuchi, H. (1995). *The knowledge-creating company: How Japanese companies create the dynamics of innovation*. Oxford University Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Polanyi, M. (1966). *The tacit dimension*. Doubleday.